Bash Shell Scripting Lab – Scripts et Automatisation sous Kali Linux

TP 2 – Scripting Shell

Karadag Nissa
Bachelor 2 Informatique – Ynov Campus
Formation : Systèmes Linux
Décembre 2025

1. Objectif

Ce TP a pour but de maîtriser les fondamentaux du scripting Bash : boucles, structures conditionnelles, interactions utilisateur, manipulation de fichiers et automatisation de tâches. L'environnement utilisé est une machine virtuelle Kali Linux sous VirtualBox.

2. Environnement de travail

Les scripts ont été développés et exécutés sur une VM Kali Linux. Le répertoire de travail est ~/tp_shell. Chaque script est créé avec nano, rendu exécutable avec chmod +x, puis lancé avec ./nom_script.sh.

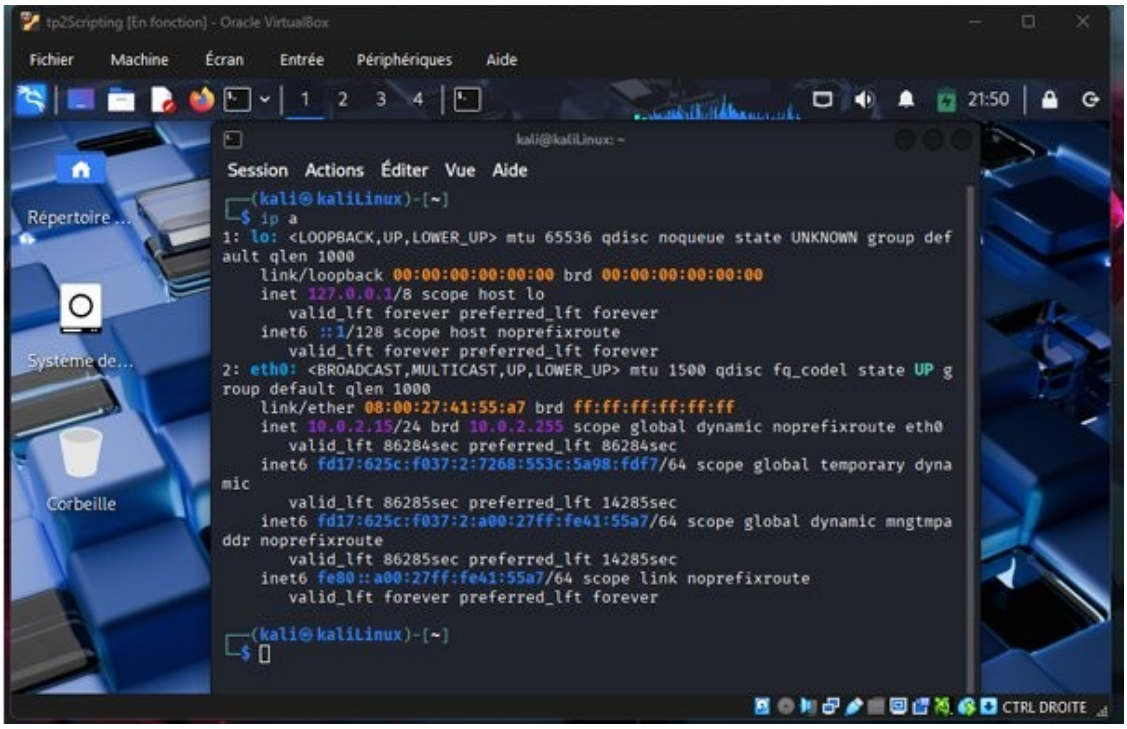

Fig. 1 – Interface réseau de la VM Kali Linux (ip a)

3. Operateurs de comparaison Bash

Les opérateurs de comparaison numériques utilisés dans les structures conditionnelles Bash :

Operateur	Signification
-lt	Strictement inférieur à (<)
-le	Inférieur ou égal à (<=)
-eq	Egal à (=)
-ne	Différent de (!=)
-gt	Strictement supérieur à (>)
-ge	Supérieur ou égal à (>=)

4. Exercice 1 – Boucle while : compter de 1 à 10

Objectif

Créer un script utilisant une boucle while pour compter de 1 à 10 et afficher chaque nombre.

Concepts

Implémentation d'une boucle while contrôlée par une variable compteur. Le script initialise i=1, évalue la condition [$i -le 10], affiche la valeur de i, puis incrémente via une expression arithmétique.

Syntaxe

```
while [ condition ]
do
    commandes
done
```

Script – ex1_while.sh

```
#!/bin/bash
# Exercice 1 : afficher les nombres de 1 à 10 avec une boucle while
i=1
while [ "$i" -le 10 ]
do
    echo "$i"
    i=$((i + 1))
done
```

Résultat

Fig. 2 – Résultat de ex1_while.sh et affichage du code source

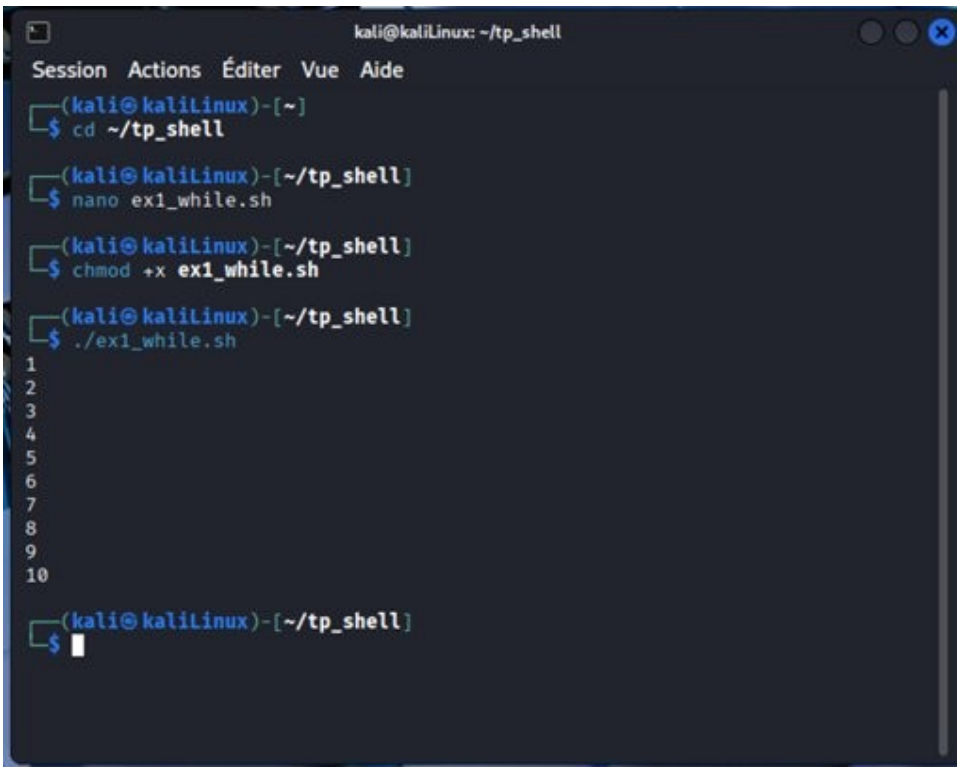

Fig. 3 – Exécution pas à pas : création, chmod +x et lancement

5. Exercice 2 – Boucle for : parcourir une liste de noms

Objectif

Créer un script utilisant une boucle for pour itérer sur une liste de noms et afficher un message personnalisé pour chacun.

Concepts

Utilisation d'une boucle for pour parcourir une liste statique de chaînes. Chaque itération affecte un élément à la variable nom et déclenche un affichage formaté.

Syntaxe

```
for var in $liste
do
    commandes
done
```

Script – ex2_for.sh

```
#!/bin/bash
# Exercice 2 : parcourir une liste de noms avec une boucle for
noms="Nissa Alice Emma Hugo David Jean Sofiane"
for nom in $noms
do
    echo "Bonjour $nom, bienvenue dans le script Bash !"
done
```

Résultat

```
kali@kaliLinux: ~
Session  Actions  Éditer  Vue  Aide
┌──(kali㉿kaliLinux)-[~]
└─$ nano ex2_for.sh

┌──(kali㉿kaliLinux)-[~]
└─$ chmod +x ex2_for.sh

┌──(kali㉿kaliLinux)-[~]
└─$ ./ex2_for.sh
Bonjour Nissa, bienvenue dans le script Bash !
Bonjour Alice, bienvenue dans le script Bash !
Bonjour Emma, bienvenue dans le script Bash !
Bonjour Hugo, bienvenue dans le script Bash !
Bonjour David, bienvenue dans le script Bash !
Bonjour Jean, bienvenue dans le script Bash !
Bonjour Sofiane, bienvenue dans le script Bash !

┌──(kali㉿kaliLinux)-[~]
└─$ cat ./ex2_for.sh
#!/bin/bash

# Exercice 2 : parcourir une liste de noms avec une boucle for

noms="Nissa Alice Emma Hugo David Jean Sofiane"

for nom in $noms
do
    echo "Bonjour $nom, bienvenue dans le script Bash !"
done
```

Fig. 4 – Résultat de ex2_for.sh : messages personnalisés pour chaque nom

```
kali@kaliLinux: ~
Session  Actions  Éditer  Vue  Aide
┌──(kali㉿kaliLinux)-[~]
└─$ chmod +x ex2_for.sh

┌──(kali㉿kaliLinux)-[~]
└─$ ./ex2_for.sh
Bonjour Nissa, bienvenue dans le script Bash !
Bonjour Alice, bienvenue dans le script Bash !
Bonjour Emma, bienvenue dans le script Bash !
Bonjour Hugo, bienvenue dans le script Bash !
Bonjour David, bienvenue dans le script Bash !
Bonjour Jean, bienvenue dans le script Bash !
Bonjour Sofiane, bienvenue dans le script Bash !

┌──(kali㉿kaliLinux)-[~]
└─$ cat ./ex2_for.sh
#!/bin/bash

# Exercice 2 : parcourir une liste de noms avec une boucle for

noms="Nissa Alice Emma Hugo David Jean Sofiane"

for nom in $noms
do
    echo "Bonjour $nom, bienvenue dans le script Bash !"
done

┌──(kali㉿kaliLinux)-[~]
└─$
```

Fig. 5 – Affichage du code source de ex2_for.sh

6. Exercice 3 – Jeu du nombre à deviner

Objectif

Créer un script Bash permettant à l'utilisateur de deviner un nombre entré manuellement entre 1 et 20. Toutes les tentatives sont enregistrées dans un fichier joueur_log.txt. Le script indique si le nombre est plus grand ou plus petit.

Concepts

Réalisation d'un mini-jeu basé sur une boucle infinie évaluant des comparaisons numériques. Gestion d'entrées utilisateur, tests conditionnels (-lt, -gt), incrémentation d'un compteur de tentatives et journalisation dans un fichier via redirection (>>).

Elements cles

- Vider un fichier : > fichier.txt
- Lire une entrée utilisateur : read -p "Message : " variable
- Ecrire dans un fichier : echo "texte" >> fichier.txt

Script – ex3_jeu.sh

```
#!/bin/bash
log="joueur_log.txt"
> "$log"
echo " Jeu nombre à deviner entre 1 et 20"
read -p "Entrez le nombre à deviner (entre 1 et 20) : " secret
if [ "$secret" -lt 1 ] || [ "$secret" -gt 20 ]; then
    echo "Le nombre doit être entre 1 et 20." && exit 1
fi
clear && echo "Le nombre secret a été choisi ! À vous de deviner."
tentative_nb=0
while true
do
    read -p "Votre proposition : " proposition
    tentative_nb=$((tentative_nb + 1))
    if [ "$proposition" -lt "$secret" ]; then
        echo "C'est plus grand !"
        echo "Tentative $tentative_nb : $proposition → trop petit" >> "$log"
    elif [ "$proposition" -gt "$secret" ]; then
        echo "C'est plus petit !"
        echo "Tentative $tentative_nb : $proposition → trop grand" >> "$log"
    else
        echo "Bravo ! Vous avez trouvé en $tentative_nb tentative(s) !"
        echo "Tentative $tentative_nb : $proposition → trouvé !" >> "$log"
        break
    fi
done
echo "Les tentatives ont été enregistrées dans le fichier : $log"
```

Résultat

```
kali@kaliLinux: ~
Session  Actions  Éditer  Vue  Aide
Le nombre secret a été choisi ! À vous de deviner.
Votre proposition : 4
C'est plus grand !
Votre proposition : 20
C'est plus petit !
Votre proposition : 14
Bravo ! Vous avez trouvé en 3 tentative(s) !
Les tentatives ont été enregistrées dans le fichier : joueur_log.txt

┌──(kali㉿kaliLinux)-[~]
└─$ nano ex3_jeu.sh

┌──(kali㉿kaliLinux)-[~]
└─$ chmod +x ex3_jeu.sh

┌──(kali㉿kaliLinux)-[~]
└─$ ./ex3_jeu.sh
 Jeu nombre à deviner entre 1 et 20
Entrez le nombre à deviner (entre 1 et 20) :
```

Fig. 6 – Partie complète : 3 tentatives, trouvé en proposant 4, 20, puis 14

```
kali@kaliLinux: ~
Session  Actions  Éditer  Vue  Aide
Les tentatives ont été enregistrées dans le fichier : joueur_log.txt

┌──(kali㉿kaliLinux)-[~]
└─$ cat ./ex3_jeu.sh
#!/bin/bash

# Exercice 3 : jeu du nombre à deviner (1 à 20) avec log des tentatives

log="joueur_log.txt"

# On vide le fichier de log au début du jeu
> "$log"

echo " Jeu nombre à deviner entre 1 et 20"
read -p "Entrez le nombre à deviner (entre 1 et 20) : " secret

# Vérification basique
if [ "$secret" -lt 1 ] || [ "$secret" -gt 20 ]; then
    echo "Le nombre doit être entre 1 et 20."
    exit 1
fi

clear
echo "Le nombre secret a été choisi ! À vous de deviner."

tentative_nb=0

while true
do
    read -p "Votre proposition : " proposition
    tentative_nb=$((tentative_nb + 1))

    if [ "$proposition" -lt "$secret" ]; then
        echo "C'est plus grand !"
        echo "Tentative $tentative_nb : $proposition → trop petit" >> "$log"
    elif [ "$proposition" -gt "$secret" ]; then
        echo "C'est plus petit !"
        echo "Tentative $tentative_nb : $proposition → trop grand" >> "$log"
    else
        echo "Bravo ! Vous avez trouvé en $tentative_nb tentative(s) !"
        echo "Tentative $tentative_nb : $proposition → trouvé !" >> "$log"
        break
    fi
done

echo "Les tentatives ont été enregistrées dans le fichier : $log"

┌──(kali㉿kaliLinux)-[~]
└─$
```

Fig. 7 – Code source complet de ex3_jeu.sh

7. Exercice 4 – Vérifier l'existence d'un fichier ou dossier

Objectif

Ecrire un script qui demande un chemin et indique s'il existe, s'il s'agit d'un fichier ou d'un dossier.

Concepts

Script de vérification d'un chemin via des tests de fichiers : -e (existence), -f (fichier régulier) et -d (répertoire). Affichage du type détecté selon les résultats des conditions.

Syntaxe if/elif/else

```
if [ condition ]; then
    commandes
elif [ condition ]; then
    commandes
else
    commandes
fi
```

Script – ex4_check_file.sh

```
#!/bin/bash
# Exercice 4 : vérifier l'existence d'un fichier ou d'un dossier
read -p "Entrez le nom d'un fichier ou d'un dossier : " chemin
if [ -e "$chemin" ]; then
    echo "Le chemin '$chemin' existe."
    if [ -f "$chemin" ]; then
        echo "Il s'agit d'un fichier."
    elif [ -d "$chemin" ]; then
        echo "Il s'agit d'un dossier."
    else
        echo "Ce n'est ni un fichier régulier ni un dossier."
    fi
else
    echo "Le chemin '$chemin' n'existe pas."
fi
```

Résultat

```
┌──(kali㉿kaliLinux)-[~]
└─$ cd tp_shell

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ nano ex4_check_file.sh


┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ chmod +x ex3_jeu.sh

chmod: impossible d'accéder à 'ex3_jeu.sh': Aucun fichier ou dossier de ce no
m

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ nano ex4_check_file.sh


┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ chmod +x ex4_check_file.sh


┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ ./ex4_check_file.sh

Entrez le nom d'un fichier ou d'un dossier : tp_shell
Le chemin 'tp_shell' n'existe pas.

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ ./ex4_check_file.sh

Entrez le nom d'un fichier ou d'un dossier : Musique
Le chemin 'Musique' n'existe pas.

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ ./ex4_check_file.sh

Entrez le nom d'un fichier ou d'un dossier : /etc
Le chemin '/etc' existe.
Il s'agit d'un dossier.

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$
```

Fig. 8 – Tests avec tp_shell (inexistant), Musique (inexistant) et /etc (dossier)

```
┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ cat ./ex4_check_file.sh

#!/bin/bash

# Exercice 4 : vérifier l'existence d'un fichier ou d'un dossier

read -p "Entrez le nom d'un fichier ou d'un dossier : " chemin

if [ -e "$chemin" ]; then
    echo "Le chemin '$chemin' existe."

    if [ -f "$chemin" ]; then
        echo "Il s'agit d'un fichier."
    elif [ -d "$chemin" ]; then
        echo "Il s'agit d'un dossier."
    else
        echo "Ce n'est ni un fichier régulier ni un dossier."
    fi
else
    echo "Le chemin '$chemin' n'existe pas."
fi

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$
```

Fig. 9 – Code source de ex4_check_file.sh

8. Exercice 5 – Script de sauvegarde simple

Objectif

Créer un script qui demande un dossier source et un dossier de destination, copie tous les fichiers du dossier source vers la destination et affiche le nombre de fichiers copiés.

Concepts

Automatisation d'une opération de sauvegarde : contrôle d'existence du répertoire source, création conditionnelle du répertoire destination (mkdir -p), parcours du contenu avec une boucle for, détection de fichiers réguliers (-f) et copie (cp). Un compteur suit le nombre de fichiers copiés.

Elements cles

- Boucle sur répertoire : for fichier in "$rep"/*
- Compteur : compteur=$((compteur + 1))
- Création récursive de dossier : mkdir -p "$dest"

Script – ex5_backup.sh

```
#!/bin/bash
# Exercice 5 : script de sauvegarde simple
read -p "Dossier source : " src
read -p "Dossier de destination : " dest
if [ ! -d "$src" ]; then
    echo "Le dossier source n'existe pas." && exit 1
fi
if [ ! -d "$dest" ]; then
    echo "Le dossier de destination n'existe pas. Création... "
    mkdir -p "$dest"
fi
compteur=0
for fichier in "$src"/*
do
    if [ -f "$fichier" ]; then
        cp "$fichier" "$dest"
        compteur=$((compteur + 1))
    fi
done
echo "Sauvegarde terminée."
echo "Nombre de fichiers copiés : $compteur"
```

Résultat et code source

```
┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ cat ./ex5_backup.sh

#!/bin/bash

# Exercice 5 : script de sauvegarde simple

read -p "Dossier source : " src
read -p "Dossier de destination : " dest

# Vérifier que le dossier source existe
if [ ! -d "$src" ]; then
    echo "Le dossier source n'existe pas."
    exit 1
fi

# Créer le dossier de destination s'il n'existe pas
if [ ! -d "$dest" ]; then
    echo "Le dossier de destination n'existe pas. Création..."
    mkdir -p "$dest"
fi

compteur=0

for fichier in "$src"/*
do
    if [ -f "$fichier" ]; then
        cp "$fichier" "$dest"
        compteur=$((compteur + 1))
    fi
done

echo "Sauvegarde terminée."
echo "Nombre de fichiers copiés : $compteur"
```

Fig. 10 – Code source de ex5_backup.sh

Test réalisé : copie de f1.txt et f2.txt depuis source_test vers dest_test. Résultat : 2 fichiers copiés.

9. Exercice 6 – Menu interactif Bash

Objectif

Créer un script affichant un menu interactif avec 4 options : afficher la date, afficher le répertoire courant, lister les fichiers, quitter. Le script tourne en boucle jusqu'à ce que l'utilisateur choisisse "Quitter".

Concepts

Construction d'un menu interactif utilisant une boucle while true et une structure case pour dispatcher les actions. Chaque option exécute une commande système (date, pwd, ls) ou provoque la sortie de la boucle (break).

Syntaxe case

```
case $variable in
    valeur1)
        commandes
        ;;
    valeur2)
        commandes
        ;;
    *)
        commandes
        ;;
esac
```

Script – ex6_menu.sh

```
#!/bin/bash
# Exercice 6 : menu interactif Bash
while true
do
    echo "===== MENU ====="
    echo "1) Afficher la date"
    echo "2) Afficher le répertoire courant"
    echo "3) Lister les fichiers"
    echo "4) Quitter"
    echo "---------------"
    read -p "Votre choix : " choix
    case "$choix" in
        1) echo "Date actuelle :" && date ;;
        2) echo "Répertoire courant :" && pwd ;;
        3) echo "Liste des fichiers :" && ls ;;
        4) echo "Au revoir !" && break ;;
        *) echo "Choix invalide, veuillez réessayer." ;;
    esac
    echo
done
```

Résultat

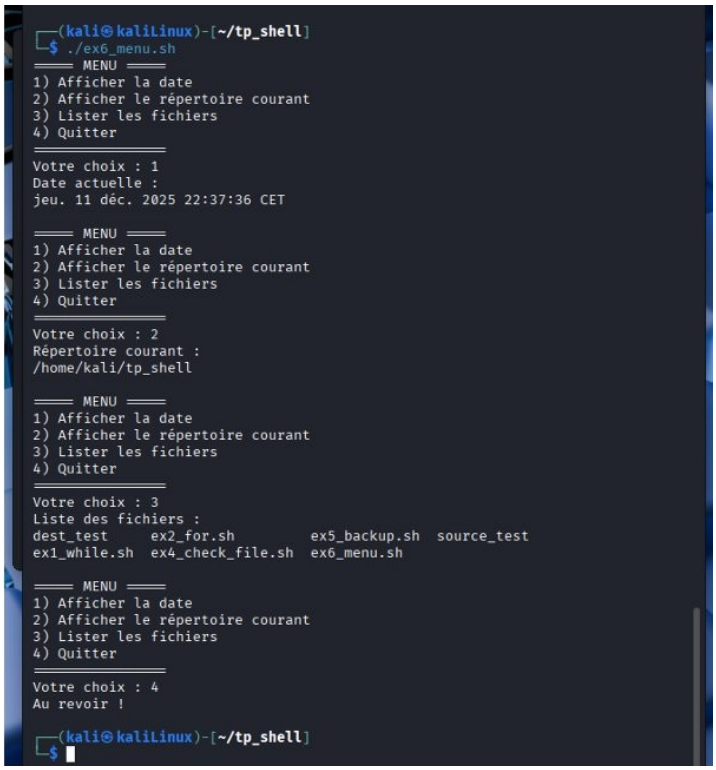

Fig. 11 – Exécution du menu : options 1 (date), 2 (pwd), 3 (ls), 4 (quitter)

```
┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$ cat ./ex6_menu.sh
#!/bin/bash

# Exercice 6 : menu interactif Bash

while true
do
    echo "═══ MENU ═══"
    echo "1) Afficher la date"
    echo "2) Afficher le répertoire courant"
    echo "3) Lister les fichiers"
    echo "4) Quitter"
    echo "═══════════"
    read -p "Votre choix : " choix

    case "$choix" in
        1)
            echo "Date actuelle :"
            date
            ;;
        2)
            echo "Répertoire courant :"
            pwd
            ;;
        3)
            echo "Liste des fichiers :"
            ls
            ;;
        4)
            echo "Au revoir !"
            break
            ;;
        *)
            echo "Choix invalide, veuillez réessayer."
            ;;
    esac

    echo
done

┌──(kali㉿kaliLinux)-[~/tp_shell]
└─$
```

Fig. 12 – Code source complet de ex6_menu.sh

10. Bilan des scripts réalisés

Exercice	Fichier	Concept clé	Statut
Exercice 1	ex1_while.sh	Boucle while, incrémentation	Réalisé
Exercice 2	ex2_for.sh	Boucle for, liste de chaînes	Réalisé
Exercice 3	ex3_jeu.sh	Boucle infinie, comparaisons, log fichier	Réalisé
Exercice 4	ex4_check_file.sh	Tests fichiers -e -f -d, if/elif/else	Réalisé
Exercice 5	ex5_backup.sh	Boucle for répertoire, cp, compteur	Réalisé
Exercice 6	ex6_menu.sh	while true, case, menu interactif	Réalisé

Conclusion

Ce TP a permis de couvrir les structures fondamentales du scripting Bash à travers six exercices progressifs. Les compétences acquises sont :

- Maîtrise des boucles while et for pour l'itération et le comptage
- Utilisation des structures conditionnelles if/elif/else et case pour la prise de décision
- Gestion des entrées utilisateur avec read -p
- Manipulation de fichiers : création, écriture avec redirection (> et >>), lecture avec cat
- Tests du système de fichiers : -e, -f, -d pour vérifier l'existence et le type d'un chemin
- Automatisation d'une sauvegarde avec boucle sur répertoire et compteur
- Construction d'un menu interactif persistant avec while true et case

Karadag Nissa – Bachelor 2 Informatique – Ynov Campus – 2025